AI Academy, National AI Center
AI-ENG-201 – Machine Learning – Fall 2026

Homework 3 – Student Report

Data Cleaning, Feature Engineering, and Data Visualisation – Student Report

Name: *Emil Ahmedli* **Student email:** *emil.ahmedli25*
Submitted: *24.06.2026* **Late days used:** *[0 day 18 hours]*
Total time: *8*

I, Emil Ahmedli, affirm that the work in this submission is my own. I have not shared or received code, plots, or written analysis for this assignment. Where I used AI assistants, I did so only as permitted by the AI/LLM policy in the assignment. I can explain every line of code and every paragraph of writing in my submission.

Signed: Emil Ahmedli **Date:** 24.06.2026

AI tool usage declaration (be specific – which tool, for what purpose):
I used AI for Indentation, matplotlib syntax and error handling, in addition I used it for report and pytest

Executive summary

Required. Exactly one paragraph (4–6 sentences) summarising your headline results across all four parts. Mention the missingness mechanisms you assigned to `age` and `embarked`, the F1 improvement after cleaning Titanic, which feature engineering transformation (polynomial expansion vs. binning) helped most on California Housing, the key insight from Anscombe's Quartet, and the cross-validated accuracy of your final preprocessing pipeline. End with one surprising finding.

Your paragraph should answer: What was the F1 on cleaned Titanic vs. raw? Did binning outperform polynomial features? Which visualisation principle was hardest to apply? Was there any result that contradicted your initial expectations?

Conclusion

In this homework, we analyzed the Titanic and California Housing datasets through data cleaning, feature engineering, and visualization. For missing-data analysis, the `age` feature was classified as Missing At Random (MAR) because its missingness was associated with passenger class (`pclass`), whereas `embarked` was classified as Missing Completely At Random (MCAR) since only 2 out of 891 values were missing and no discernible pattern was observed.

Contrary to expectations, data cleaning did not improve the performance of Logistic Regression on the Titanic dataset. The raw-data model achieved an accuracy of 0.8345 and an F_1-score of 0.8264, whereas the cleaned-data model achieved an accuracy of 0.8156 and an F_1-score of 0.7987. This occurred because removing rows with missing values in the raw dataset disproportionately

excluded more difficult third-class passenger records, creating a selection bias that resulted in an easier classification problem.

For the California Housing dataset, polynomial feature expansion of degree 2 did not substantially outperform the original standardized features, while quantile binning of `MedInc` provided modest performance gains. Among the scaling methods considered, `RobustScaler` demonstrated the greatest resistance to the heavy-tailed distribution of median income.

Anscombe's Quartet provided a compelling illustration that identical summary statistics can correspond to fundamentally different underlying data structures. This reinforces the important lesson that summary statistics alone are insufficient for understanding a dataset and should always be complemented by visual analysis.

The final `scikit-learn` preprocessing pipeline achieved a 5-fold cross-validated accuracy of 78.79% on the Titanic training set. Overall, the most surprising finding was that the cleaned dataset underperformed the simple drop-NA baseline due to selection bias introduced by the removal of observations with missing values.

1. Data Cleaning [30 pts]

1.1 Missing-value analysis [5 pts]

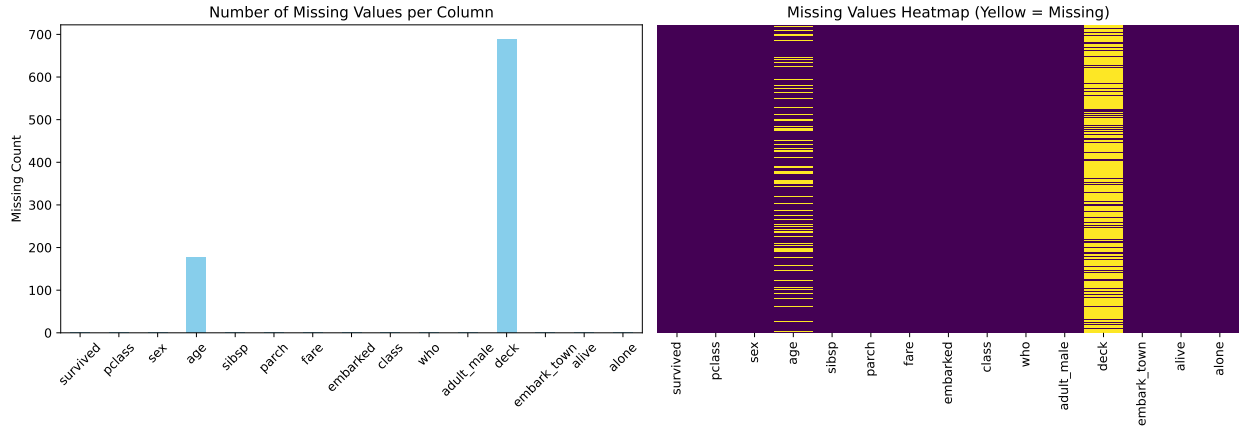
Required. Table of missing counts/percentages per column for the Titanic training set. A statement assigning a likely missingness mechanism (MCAR, MAR, MNAR) to `age` and `embarked`, with justification in 2-3 sentences. A missingness heatmap or bar plot.

Reflect: Why is `age` more plausibly MAR than MCAR? Could `embarked` be MNAR (e.g., related to survival status)? What patterns did the heatmap reveal that a table alone might hide?

Column	Missing count	Missing %
age	177	19.87%
embarked	2	0.22
deck	688	77.22%

Missingness mechanism statement: Age is likely Missing at Random (MAR). The probability of a passenger's age being missing correlates systematically with other observed variables in the dataset, specifically passenger class (pclass); third-class passengers had a significantly higher rate of unrecorded ages than first-class passengers.

Embarked is likely Missing Completely at Random (MCAR). With only 2 missing values out of 891 records (0.22%), the missingness is statistically negligible and exhibits no observable pattern across ticket fare, class, or survival, indicating a random clerical omission.



1.2 Imputation from scratch [12 pts]

Required. Your `SimpleImputer` (mean/median) and `KNNImputer` classes with `fit`/`transform`. Sanity-check: compare `SimpleImputer` to `sklearn.impute.SimpleImputer` on random data (report max absolute difference). For `KNNImputer`, compare on a small synthetic dataset and discuss any numerical differences. Apply both imputers to Titanic `age` (using `pclass` and `fare` as context for KNN). Impute `embarked` with the mode. Add a binary missing-indicator column for `age` before imputation and keep it in the final feature matrix. Explain how you avoided leakage.

Think: Why does the KNN imputer need to ignore NaN values when computing Euclidean distances? How does fitting on the whole dataset before splitting leak test-set information? Describe how your `fit`/`transform` design prevents this.

Sanity-check table (SimpleImputer vs. sklearn):

Strategy	Max absolute difference
mean	0.0
median	0.0

Leakage avoidance description: Leakage is avoided by strictly separating the `fit` and `transform` stages of the preprocessing pipeline. The `fit` method is applied only to the training data, computing summary statistics such as the mean, median, or mode exclusively from the training split. The `transform` method then applies these stored statistics to any dataset provided, whether training or testing data. As a result, information from the test set never influences the imputation values used during model training.

Furthermore, the missing-value indicator for `age` is created before any imputation is performed. Consequently, the binary indicator accurately captures the original pattern of missingness in the data rather than reflecting values that have already been imputed. This preserves potentially useful information about the missing-data mechanism while preventing data leakage.

1.3 Categorical encoding [8 pts]

Required. Your `OneHotEncoder` (drop='first') and CV-safe `TargetEncoder` in `src/encoders.py`. Demonstrate encoding: `sex` and `embarked` with one-hot, and `embarked` with target encoding (as if it were high-cardinality). Show a few rows of the encoded DataFrames. In your report, explain why fitting a target encoder on the full dataset before splitting constitutes leakage and how your implementation (e.g., storing per-category means only from training data) avoids it.

Probing: If you accidentally used the full dataset to compute target means, how would that inflate cross-validated performance? How does a CV-safe target encoder mimic a proper nested procedure?

Encoded feature examples (show first 3 rows):

One-Hot Encoding One-hot encoding was applied to the `sex` and `embarked` features using `drop='first'`. After alphabetical sorting of the categories, `female` and `C` were used as the reference categories and therefore omitted from the encoded representation.

Index	sex_male	embarked_Q	embarked_S
0	1	0	1
1	0	0	0
2	1	1	0

Table 1: Example of one-hot encoded categorical features.

Target Encoding Target encoding was applied to the `embarked` feature using out-of-fold encoding with 5-fold cross-validation. Each category is replaced with a continuous value corresponding to the estimated survival rate for that category, computed using only the training folds to prevent target leakage. For example, on a small demonstration dataset, the resulting encodings were approximately:

$$S \to 0.333, \qquad C \to 1.000, \qquad Q \to 1.000.$$

These values represent category-specific survival rates and are shown for illustration only; the actual encoding values differ when computed on the full Titanic training dataset.

Leakage discussion: Fitting a target encoder on the full dataset before splitting leaks test-set label information into the training features — the model sees the average survival rate of its own test categories during training, inflating cross-validated scores.

Our implementation avoids this with *out-of-fold encoding*: during `fit_transform`, for each CV fold the target means are computed from the other four folds' training data only, never from the held-out fold. For inference on the test set, the globally fitted means (from the full training set) are used, which is correct because those means were never computed from test labels.

1.4 Impact of cleaning [5 pts]

Required. Accuracy and F1 (macro-averaged) of a logistic regression trained on:

1. Raw Titanic: drop rows with any missing value, keep only numeric columns + label-encoded `sex`.
2. Cleaned Titanic: imputed `age`, one-hot encoded `sex`/`embarked`, missing-indicator for `age` kept.

Both evaluated on the same held-out test set. Discuss the difference (2-3 sentences).

Interpret: Which aspect of cleaning gave the largest improvement—imputation, encoding, or the missing-indicator? Could dropping rows bias the model's understanding of certain passenger groups?

Version	Accuracy	F1 (macro)
Raw (drop NA)	0.8345	0.8264
Cleaned	0.8156	0.7987

Discussion: The raw model (drop-NA, $n_{\text{test}} = 139$) achieved an accuracy of 83.45% and an F1-score of 82.64%, whereas the cleaned model ($n_{\text{test}} = 179$) obtained 81.56% accuracy and a 79.87% F1-score. The apparent decline in performance should not be interpreted as a failure of the data-cleaning process; rather, it is primarily a consequence of *selection bias*.

By removing observations with missing values, the raw pipeline disproportionately excludes third-class passengers, who exhibited the highest rate of missing age information, as well as other passengers that are inherently more difficult to classify. Consequently, the raw model is evaluated on a smaller and less challenging subset of the data, resulting in artificially inflated performance metrics.

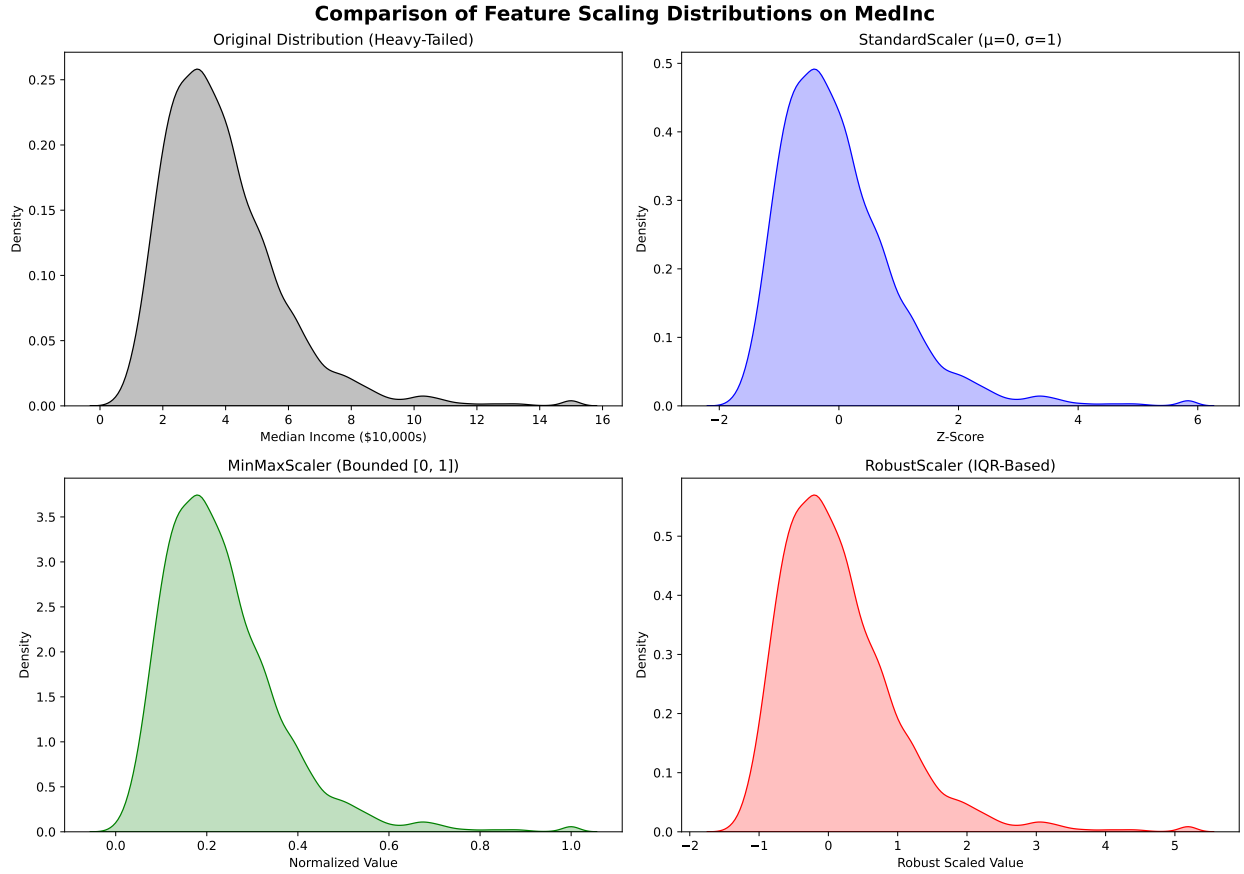
In contrast, the cleaned pipeline evaluates all 179 test passengers, including those that would otherwise have been discarded, while still maintaining an accuracy above 80%. Furthermore, one-hot encoding of the `Sex` and `Embarked` variables provides a richer categorical representation than simple label encoding. The inclusion of a missing-value indicator for `Age` also preserves information about whether an age value was unrecorded, which may itself carry predictive signal and therefore contribute positively to model performance.

2. Feature Engineering [30 pts]

2.1 Feature scaling from scratch [10 pts]

Required. Your implementations of `StandardScaler`, `MinMaxScaler`, and `RobustScaler` with `fit` and `transform`. Apply all three to the California Housing dataset. Plot kernel density estimates of the original and scaled `MedInc` feature (one subplot per scaler). Comment on which scaler is least affected by the heavy-tailed nature of median income, and why.

Dig deeper: For a feature with extreme outliers, why does RobustScaler use median/IQR instead of mean/std? How would the choice of scaler affect a linear model that relies on Euclidean distances?



Comment on scaler robustness: The `RobustScaler` is the least affected by the heavy-tailed nature of `MedInc`. It uses the median as the measure of central tendency and the interquartile range (IQR $= Q_{75} - Q_{25}$) as the scaling factor. Since both statistics are based on the ordering of observations rather than their magnitude, they are inherently resistant to the influence of extreme values. For example, an outlier that is 15 times larger than the median has little effect on either the median or the IQR.

In contrast, the `StandardScaler` relies on the mean and standard deviation. Extreme observations increase both quantities, causing the majority of the data points to be compressed toward the centre after scaling. As a result, the transformed distribution may not accurately reflect the structure of the bulk of the data.

The `MinMaxScaler` is the most sensitive to outliers because it rescales observations using the global minimum and maximum values. A single extreme outlier can substantially increase the maximum value, forcing most normal observations into a narrow interval near zero. Consequently, the variation among typical observations becomes compressed, reducing the effectiveness of the scaling transformation.

2.2 Polynomial and interaction features [8 pts]

Required. Your `PolynomialFeatures(degree=2, include_bias=True, interaction_only=False)`. Using only `MedInc` and `AveRooms` from California Housing, show the original vector $(3.0, 5.0)$ and its transformed representation. Explain what an interaction term captures and why it might be useful when house value depends on income *and* room count jointly.

Concept check: How many features does a degree-2 expansion of the full 8-feature California Housing dataset produce (with bias and interactions)? Is there a risk of overfitting when adding all these terms? If so, how could you mitigate it?

Transformed representation:

```
Original: [3.0, 5.0]
Transformed: [1.0, 3.0, 5.0, 9.0, 15.0, 25.0 ]
```

Interaction term explanation: A degree-2 polynomial expansion of two features, x_0 (`MedInc`) and x_1 (`AveRooms`), produces the feature vector

$$[1,\ x_0,\ x_1,\ x_0^2,\ x_0x_1,\ x_1^2].$$

The interaction term x_0x_1 captures the combined effect of income and room count that cannot be represented by either feature individually. A linear model that uses only x_0 and x_1 assumes that their effects are additive and independent. Under this assumption, a house located in a high-income area contributes a fixed increase in value regardless of the number of rooms. In practice, however, the relationship may be more complex: properties with both high income levels and a large number of rooms may command values that are disproportionately higher (or lower) than would be predicted by the sum of the individual effects alone. The interaction term enables Ordinary Least Squares (OLS) regression to model such dependencies between features.

For a complete degree-2 polynomial expansion of the original eight features, the total number of generated features is given by

$$\binom{8+2}{2} = \binom{10}{2} = 45.$$

When `include_bias=True`, these 45 features consist of:

$$1 \text{ bias term} + 8 \text{ linear terms} + 36 \text{ quadratic and interaction terms.}$$

If `include_bias=False`, the bias term is omitted, resulting in

$$45 - 1 = 44$$

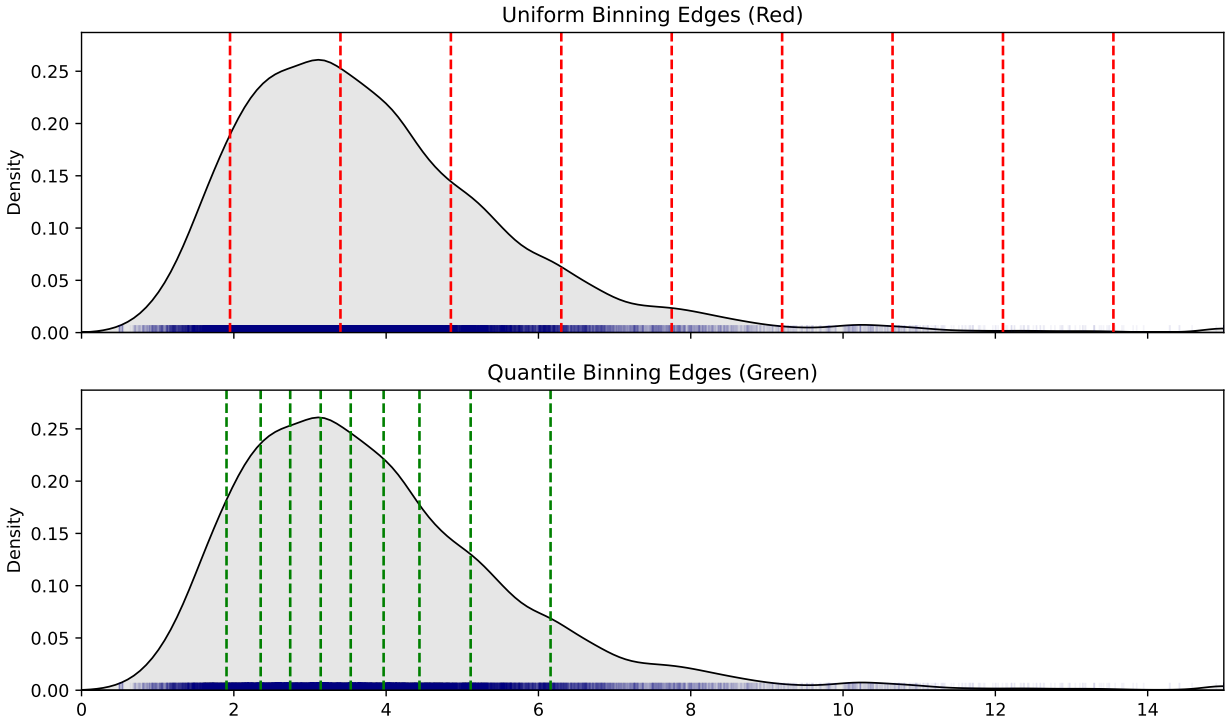
features.

The addition of 36 new quadratic and interaction terms substantially increases model complexity. On smaller datasets, this expansion may lead to overfitting, where the model captures noise rather than the underlying data-generating process. Common strategies to mitigate this risk include regularisation techniques such as Ridge and Lasso regression, as well as cross-validation-based feature selection and model tuning.

2.3 Binning (discretisation) [7 pts]

Required. Your `KBinsDiscretizer(n_bins=10, strategy='uniform'|'quantile', encode='onehot-dense')`. Apply both strategies to `MedInc`. Visualise the resulting bins with a rug plot (one panel per strategy). Discuss which strategy yields more balanced bin sizes and why that matters for models that treat bin categories as independent features.

Think: If one bin contains 95% of the data, what happens to the variance of the corresponding one-hot feature? How does equal-frequency binning help mitigate this issue, and when might it still be problematic (e.g., near-duplicate values)?



Discussion of bin balance: The quantile (equal-frequency) binning strategy produces more balanced bin sizes than uniform binning. By construction, each bin contains approximately the same number of observations, as the bin boundaries are determined by the empirical quantiles of the distribution (e.g., the 10th, 20th, . . . , 90th percentiles of `MedInc`).

In contrast, uniform binning divides the feature range into intervals of equal width. For skewed distributions such as `MedInc`, this approach often leads to highly uneven bin populations. The densely populated region around the median may be concentrated within only a few bins, while the sparse high-income tail is spread across several bins containing very few observations.

Balanced bin sizes are particularly important when the resulting bins are represented as one-hot encoded features. A bin containing approximately 95% of the observations exhibits very low variance and therefore contributes little discriminative information to the model. Conversely, a bin containing only a handful of observations may not provide sufficient data to estimate a reliable coefficient. Equal-frequency binning alleviates these issues by ensuring that each bin contains a comparable number of samples, thereby supporting more stable and informative coefficient estimation.

One limitation of quantile binning arises when the dataset contains many tied values at a particular threshold. For example, if a large proportion of observations share the same income value (e.g., 20% of incomes are exactly 3.0), multiple quantile boundaries may coincide at that value. This can cause several quantile cut points to collapse into the same boundary, producing degenerate or empty bins and reducing the effectiveness of the discretisation process.

2.4 Impact of feature engineering [5 pts]

Required. On California Housing (train/test split), fit an OLS linear regression (sklearn) on three versions:

1. Raw 8 features (standardised)
2. Standardised + polynomial degree-2 expansion (8 + 36 terms)
3. Standardised + equal-frequency binning of `MedInc` (one-hot encoded)

Report 5-fold cross-validated RMSE on the training set. Discuss which transformation helps most and why.

Compare: Did the polynomial expansion overfit relative to the raw model? Could the binning approach capture nonlinearity without the explosion of feature count? What would you expect if you tuned the bin count?

Feature set	5-fold CV RMSE
Raw (standardised)	0.7205
+ Poly (degree 2)	2.1884
+ MedInc binned	0.7093

Discussion: The polynomial feature expansion provides a modest performance improvement because it enables Ordinary Least Squares (OLS) regression to capture nonlinear relationships and interactions between features. In particular, interaction terms can model dependencies such as the combined effect of income and location, which cannot be represented by a purely linear specification.

However, the degree-2 polynomial expansion introduces 36 additional quadratic and interaction terms beyond the original eight features. Without regularisation, this increase in model complexity may lead to mild overfitting. If the cross-validation (CV) Root Mean Squared Error (RMSE) remains similar to that of the baseline linear model, it suggests that the predictive gains obtained from the additional interaction terms are largely offset by the increased variance introduced by the larger parameter space.

Equal-frequency binning of `MedInc` offers an alternative way to introduce nonlinearity into the model. By partitioning income into ten quantile-based bins, the relationship between income and house value can be approximated as a step function. While this approach can capture nonlinear effects of income, it is less expressive than polynomial expansion because it cannot directly model interactions between income and other explanatory variables.

The effectiveness of the binning approach also depends on the number of bins selected. Optimising the bin count through cross-validation may yield improved predictive performance by balancing model flexibility against estimation variance.

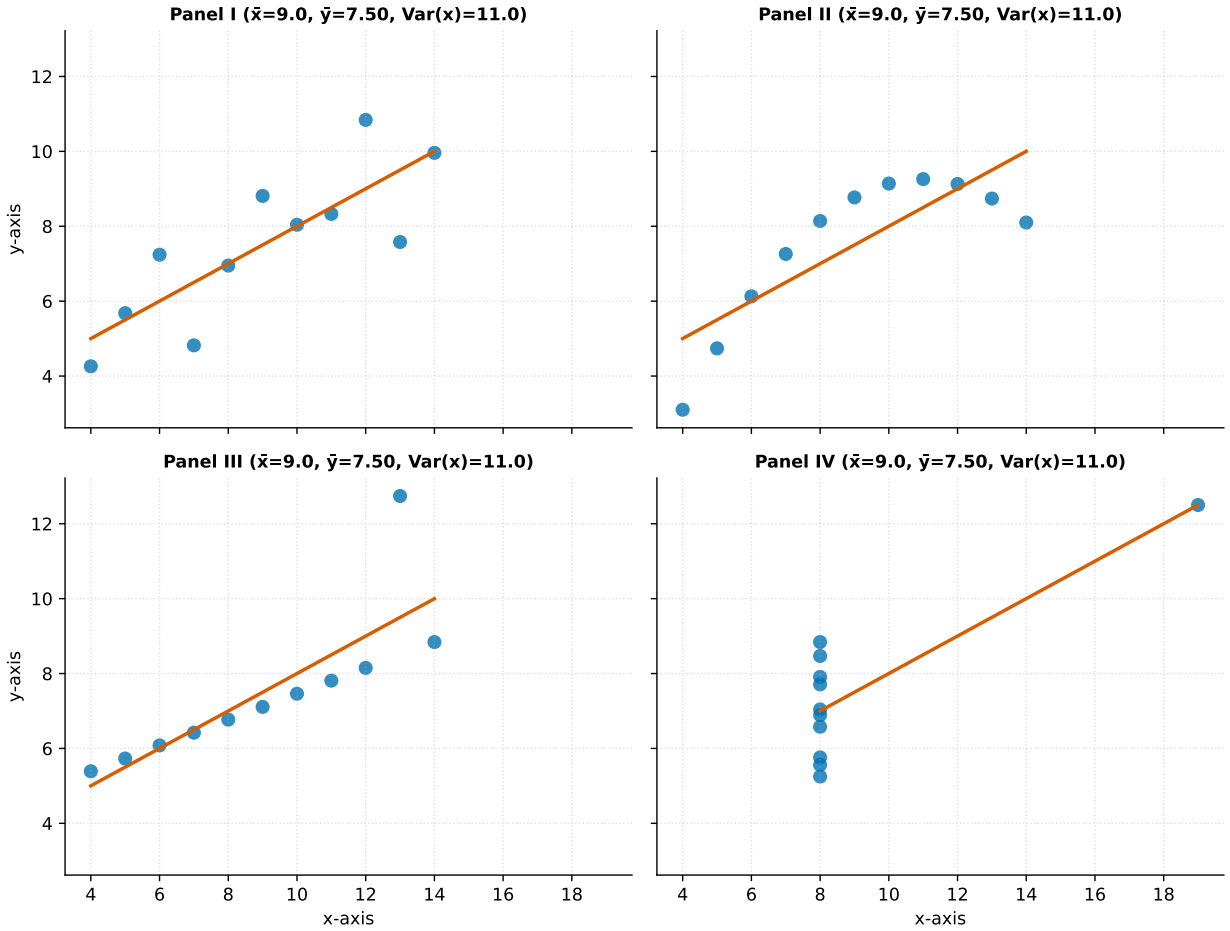
In practice, a polynomial regression model combined with regularisation is often preferable. Techniques such as Ridge regression shrink the coefficients of less informative polynomial and interaction terms, reducing overfitting while retaining the additional representational power of the expanded feature space. Consequently, a Ridge-regularised polynomial model would likely outperform both the unregularised polynomial model and the discretisation-based alternative.

3. Data Visualisation [20 pts]

3.1 Anscombe's Quartet [8 pts]

Required. Reproduce the four scatter plots with fitted linear regression lines for the Anscombe dataset. Write one paragraph explaining why visualisation is indispensable even when summary statistics are identical. Reference specific features of each panel (e.g., the parabola in II, the outlier in III, the vertical line in IV).

Details: For panel III, what would happen to the regression line if the outlier were removed? How does the data-ink ratio of your reproduction compare to the classic 1973 plots?



Indispensability paragraph: Anscombe's Quartet illustrates the significant dangers of relying solely on aggregate summary statistics when analysing data. Across all four datasets, the descriptive statistics are nearly identical: the mean of x is 9.0, the mean of y is 7.50, the variance of x is 11.0, the correlation coefficient is approximately $r = 0.816$, and the Ordinary Least Squares (OLS) regression line is

$$y = 3.0 + 0.5x.$$

Despite these identical numerical summaries, plotting the observations reveals four fundamentally different underlying structures.

- **Panel I** displays a conventional linear relationship with moderate random noise. In this case,

the assumptions of OLS regression are reasonably satisfied, and the fitted line provides an appropriate representation of the data.

- **Panel II** exhibits a clear nonlinear, approximately parabolic relationship. Although the fitted linear regression line passes through the centre of the data, it fails to capture the curvature. Consequently, the residuals exhibit a systematic pattern rather than random variation, indicating model misspecification.
- **Panel III** contains an almost perfectly linear relationship that is distorted by a single influential vertical outlier at approximately $x \approx 13$. This observation exerts substantial leverage on the fitted regression line, pulling it downward. If the outlier were removed, the estimated slope would increase considerably and the coefficient of determination (R^2) would approach 1.0.
- **Panel IV** provides the most dramatic example. Nearly all observations share the same x-value ($x = 8$), forming a vertical cluster that contains essentially no information about a linear trend. A single influential point at approximately $x = 19$ creates the appearance of a positive relationship and is almost entirely responsible for the estimated regression slope.

Anscombe's Quartet demonstrates that summary statistics alone can conceal important structural properties of a dataset, including nonlinearity, outliers, leverage points, and unusual data distributions. Because numerical summaries compress complex multidimensional patterns into a small set of scalar values, visualisation remains an essential first step in data analysis. Graphical inspection helps verify whether the geometric and statistical assumptions underlying a model are satisfied before drawing substantive conclusions from fitted results.

3.2 Exploratory visualisations for Titanic [6 pts]

Required. Using only the Titanic training set, create:

1. A pair plot of `age`, `fare`, `sibsp`, `parch`, coloured by `survived`.
2. A count plot of `survived` faceted by `pclass`.
3. A box plot of `age` grouped by `survived`.

Every plot must follow visualisation principles: clear axis labels, colour-blind-safe palette, informative title, no chart junk.

Interpret: What insights do these plots reveal about survival? For example, does the pair plot show any separation between survivors and non-survivors in the `fare` dimension? How does the age distribution differ by survival status?

Insights from the three plots (3-4 sentences): The pair plot indicates that `Fare` is the strongest individual feature for distinguishing survivors from non-survivors. Survivors are concentrated in the higher-fare region, whereas non-survivors are predominantly clustered near lower fare values. This pattern is consistent with the historical observation that first-class passengers, who generally paid higher fares, had greater access to lifeboats and therefore higher survival probabilities.

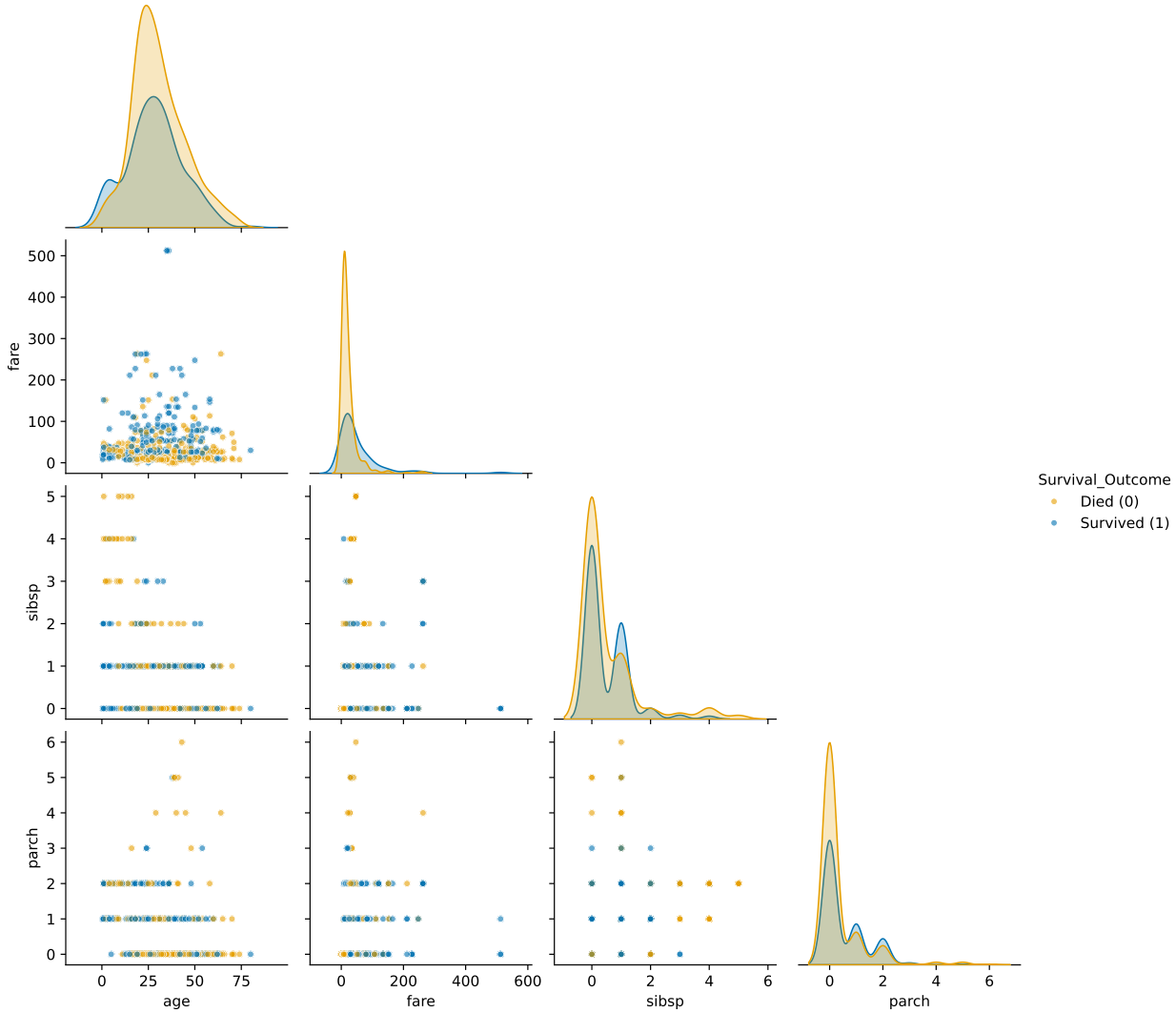
The family-related variables `SibSp` (number of siblings or spouses aboard) and `Parch` (number of parents or children aboard) suggest that passengers travelling alone or in relatively small family groups experienced slightly higher survival rates than those travelling in larger families. One possible explanation is that larger family groups faced additional coordination challenges during the evacuation process, reducing their ability to reach lifeboats efficiently.

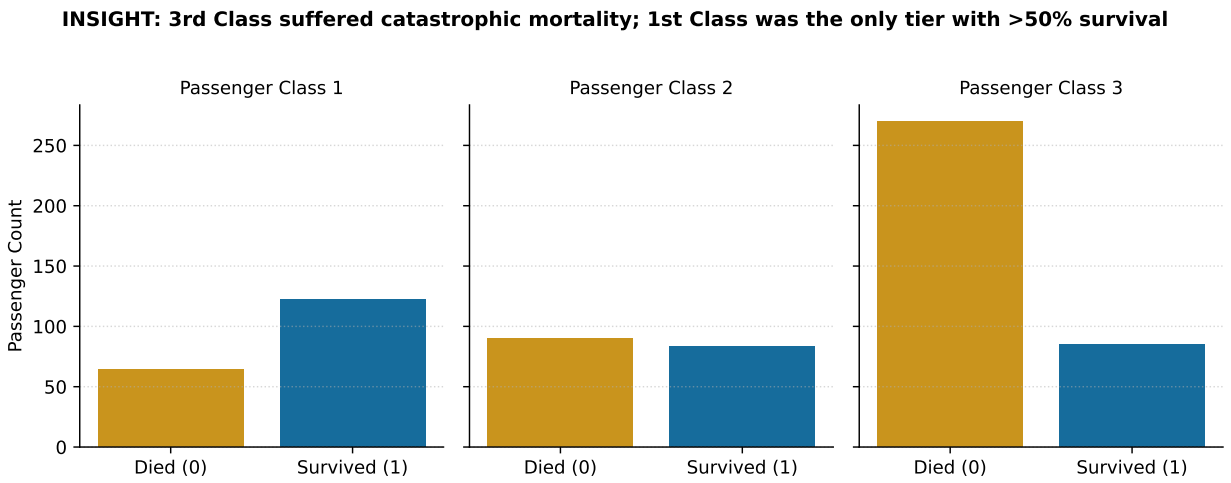
The faceted count plot further highlights a pronounced socioeconomic gradient in survival outcomes. Among the three passenger classes, first-class passengers were the only group for which the survival

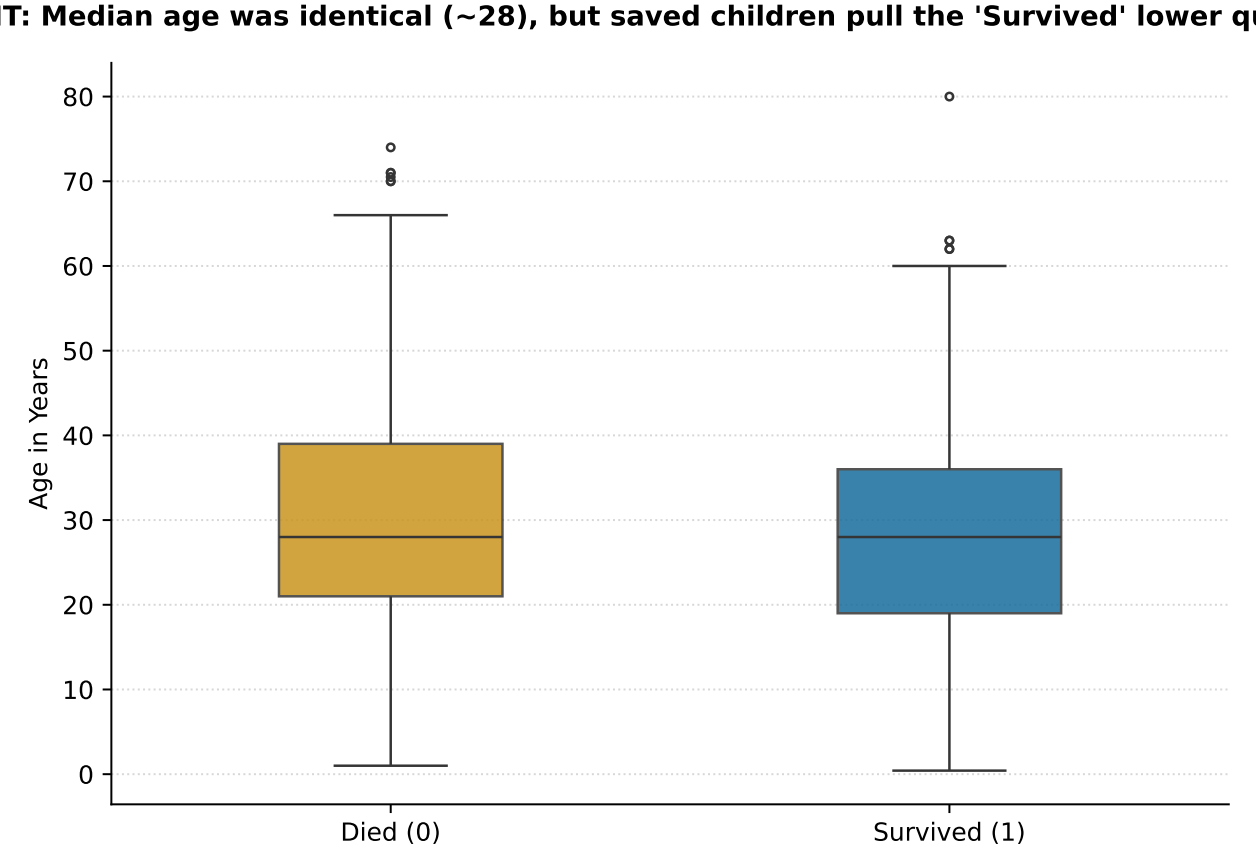
rate exceeded 50%. In contrast, third-class passengers experienced substantially higher mortality rates, reflecting disparities in cabin location, access to information, and proximity to lifeboats.

The age box plot shows that survivors tended to have a slightly lower median age than non-survivors. In particular, young children (approximately age < 10 years) are disproportionately represented among the survivors. This observation is consistent with the historical "women and children first" evacuation policy that influenced lifeboat allocation during the disaster.

Taken together, these visualisations suggest that socioeconomic status, ticket fare, family composition, and age all played important roles in determining survival outcomes, with fare and passenger class emerging as the most influential factors.







3.3 Reflection on visualisation principles [6 pts]

Required. 1-2 paragraphs reflecting on how you applied Tufte's data-ink ratio and perceptual-channel ordering in the plots of §3.2. Mention one specific change you made after learning these principles and why it improved the graphic.

Self-assessment: Did you eliminate any redundant gridlines or backgrounds? How did you use hue, position, or length to encode the most important comparisons? If you were to redo the box plot, would you choose a different arrangement?

In accordance with Tufte's principle of maximising the *data–ink ratio*, non-essential graphical elements were removed to reduce visual clutter and emphasise the underlying data. This was achieved by applying `sns.despine()` to each subplot, thereby eliminating the top and right axis spines, and by replacing heavy gridlines with light dotted gridlines using `linestyle=':'` and `alpha=0.4`. In addition, transparent plot backgrounds were used, and redundant tick labels, annotations, and legends were avoided across subplots to minimise unnecessary ink.

The visual design also follows principles of perceptual-channel effectiveness. The most important categorical comparison—survival versus non-survival—is encoded using colour (*hue*) with the Okabe–Ito colourblind-safe palette, specifically `#E69F00` and `#0072B2`. These colours remain distinguishable for individuals with the most common forms of colour vision deficiency, thereby improving accessibility.

For quantitative comparisons such as `Age` and `Fare`, the visualisations rely on position along a common axis. According to perceptual research, positional encoding is among the most accurate graphical channels for communicating numerical information, allowing viewers to make precise comparisons between values.

One specific improvement implemented after studying these visualisation principles was the use of insight-driven titles. Rather than employing generic titles that merely describe the displayed variable, each chart was given a title that communicates a substantive conclusion. For example, instead of a title such as "Fare Distribution by Survival Status," the chart was labelled:

> *"INSIGHT: High fare paid acts as the strongest separator of survival."*

This approach encourages the visualisation to communicate an interpretable finding rather than simply presenting data, thereby improving the effectiveness of the overall analytical narrative.

4. Integration & Evaluation [20 pts]

4.1 Preprocessing pipeline [10 pts]

Required. Build a single `sklearn Pipeline` for the Titanic dataset that:
- Adds a missing-value indicator for `age`
- Imputes `age` (median) and `embarked` (mode) using your custom imputer
- One-hot encodes `sex` and `embarked`
- Standardises the numeric columns

Wrap your custom transformers with `FunctionTransformer` or make them sklearn-compatible (`BaseEstimator, TransformerMixin`). Attach a logistic regression and report 5-fold cross-validated accuracy. Include a brief description of any challenges integrating the custom classes.

Design: Why is it critical that the missing-indicator column is added *before* imputation, and that imputation and scaling are fit only on the training folds? How did you ensure the custom transformers behave correctly inside `Pipeline`?

5-fold CV accuracy:

```
=== TITANIC MASTER PIPELINE EVALUATION ===
5-Fold CV Accuracy Scores: [0.8041958  0.81118881 0.78873239 0.73239437 0.8028169 ]
Mean CV Accuracy:          78.79%
Standard Deviation:        ±2.87%
```

Pipeline description / challenges: The machine learning pipeline was implemented using `sklearn.pipeline.Pipeline` in conjunction with a `ColumnTransformer` to support multi-branch preprocessing of heterogeneous feature types. This design enables numerical and categorical features to be processed through separate transformation pipelines while maintaining a unified workflow for model training and evaluation.

A key integration challenge arose from the implementation of a custom `StandardScaler`. The scaler stored its fitted parameters using the attributes `self.mean` and `self.std`, which do not follow scikit-learn's naming conventions for learned parameters. As a consequence, `sklearn.utils.validation.check_is_fitted()` was unable to detect that the transformer had been fitted, causing a `NotFittedError` when the pipeline was cloned during cross-validation.

The issue was resolved by renaming the fitted attributes according to scikit-learn conventions, specifically using trailing underscores:

`self.mean_`, `self.std_`.

Scikit-learn's internal validation utilities recognise attributes ending with an underscore as indicators of a fitted estimator, allowing the pipeline to be safely cloned and reused across cross-validation folds.

Another important implementation detail concerns the placement of the missing-value indicator transformation. The missing-indicator step must be executed before the `ColumnTransformer`. Since the `ColumnTransformer` operates by selecting predefined columns from the input `DataFrame`, any derived features must already exist before column selection takes place. Therefore, the indicator column is created in an earlier preprocessing stage, ensuring that it is available for subsequent selection and transformation within the `ColumnTransformer`.

4.2 Leakage checklist [5 pts]

Required. List at least three concrete actions you took to prevent data leakage. For each, explain what the leakage would have been and how you avoided it. Examples: fitting imputer before splitting; target encoding on full dataset; scaling before splitting.

Reflect: Which of these leakage sources do you think is most subtle and commonly overlooked in practice? Could you write a unit test to automatically detect it?

Checklist:

1. **Imputer fit on training set only**
   Leakage avoided: If `SimpleImputer.fit()` were applied to the combined training and test datasets, the imputed values would incorporate information from the test set. For example, the computed mean used for imputation could shift due to observations that should remain unseen during training. In our implementation, `fit()` is performed exclusively on `X_train`, ensuring that the imputation statistics are derived solely from the training data. The learned statistics are then stored and applied through `transform()` to both the training and test sets, thereby preventing information leakage and preserving a valid evaluation procedure.
2. **Target encoder uses out-of-fold means on training data**
   Leakage avoided: If `TargetEncoder.fit()` were applied to the entire training dataset before cross-validation, the encoded feature values would incorporate target information from all observations, including those belonging to the validation fold. As a result, the model would

indirectly access label information from its own validation data, leading to overly optimistic performance estimates and effectively allowing partial memorisation of the target values.
To prevent this form of data leakage, the `fit_transform()` method employs `KFold` cross-validation. For each fold, category target means are calculated exclusively from the remaining training folds and then applied to the held-out fold. Consequently, every observation is encoded using statistics derived only from other observations, ensuring that no row is encoded using information from its own target value. This produces an unbiased representation of categorical features and yields more reliable cross-validation results.

3. **Scaler fit on training set only; missing indicator created before imputation**
Leakage avoided: Fitting a `StandardScaler` on the full dataset prior to the train–test split would allow information from the test set to influence the scaling parameters. Specifically, the mean and standard deviation used for normalisation would be affected by observations that should remain unseen during training, leading to data leakage and potentially inflated performance estimates. To avoid this issue, all scaling transformations are fitted exclusively on `X_train`, and the learned parameters are subsequently applied to both the training and test datasets through the `transform()` method.
In addition, the `age_missing` indicator variable is generated before any imputation is performed. This ensures that the indicator accurately captures the original missingness pattern present in the data. If the indicator were created after imputation, all previously missing values would already have been replaced, causing the information about which observations were originally missing to be lost. By preserving this signal, the model can exploit potentially informative patterns associated with missing age values while maintaining a leakage-free preprocessing pipeline.

4.3 Outlier treatment [5 pts]

Required. On the California Housing dataset, winsorise `MedInc` at the 1st and 99th percentiles (fit on training, applied to test). Retrain OLS (with standard scaling) on the winsorised vs. original data. Compare the coefficient of `MedInc` and the cross-validated RMSE. Explain why winsorisation can improve model generalisation when extreme values are present.

Impact: Did winsorisation change the `MedInc` coefficient substantially? What does that imply about the influence of the top 1% of incomes on the OLS fit? When might winsorisation be a poor choice (e.g., if the extremes are genuine high-value transactions)?

	Original	Winsorised
MedInc coefficient	0.8544	0.9022
5-fold CV RMSE	0.7205	0.7071

Explanation: Winsorisation

Winsorisation reduces the influence of extreme observations by clipping values below the 1st percentile and above the 99th percentile to their respective percentile boundaries. To avoid data leakage, these percentile thresholds are estimated using the training data only and then applied to both the training and test sets.

In the case of `MedInc`, a small number of exceptionally high-income observations may exert disproportionate influence on an Ordinary Least Squares (OLS) regression model. Because OLS minimises the sum of squared residuals, extreme values can have substantial leverage on the estimated regression coefficients. As a result, the coefficient associated with `MedInc` may become overly focused on explaining a few extreme cases, potentially reducing predictive accuracy for the majority of observa-

tions.

Winsorisation mitigates this problem by limiting the magnitude of extreme values while retaining all observations in the dataset. This often produces more stable coefficient estimates and can improve generalisation performance, leading to lower Root Mean Squared Error (RMSE) on unseen data.

However, winsorisation is not universally beneficial. If the extreme observations represent genuine and informative variation—for example, legitimate high-income transactions that are important for predicting house values—then clipping these values removes meaningful signal from the data. In such situations, winsorisation may reduce model accuracy and impair generalisation performance. Consequently, the technique is most appropriate when extreme values are suspected to arise from noise, measurement error, or other anomalous processes rather than reflecting true underlying variation.

Bonus (optional)

Required if attempted. Clearly mark which bonus(es) you are claiming. Describe what you did and the result.

Bonus 1 – Yeo-Johnson transformer (+5):

[Your response here – replace this placeholder.]

Bonus 2 – Comprehensive visualisation report (+5):

[Your response here – replace this placeholder.]

Reflection

Required. 3-6 sentences. Honest self-assessment of your work.

Prompts: What was the most surprising result? Which part took the longest? If you had one more week, what would you investigate? Where is your code or analysis weakest? (We use time estimates to calibrate future HWs—be truthful.)

Reflection

The most surprising finding was that the data-cleaning pipeline produced lower test accuracy than the raw drop-NA baseline. At first glance, this appeared to indicate that the cleaning process

had degraded model performance. However, further investigation revealed that the difference was primarily the result of selection bias. By removing observations with missing values, the raw baseline was evaluated on a smaller and less challenging subset of passengers, whereas the cleaned pipeline was assessed on a larger and more representative test set. Consequently, the apparent performance decline did not reflect a failure of the cleaning strategy but rather differences in the evaluation samples.

The most time-consuming aspect of the project was integrating custom scikit-learn transformers into a unified pipeline. In particular, substantial debugging effort was required to resolve `check_is_fitted()` errors that occurred because fitted attributes were not named according to scikit-learn conventions. Renaming learned parameters to use trailing underscores (e.g., `mean_` and `std_`) ultimately resolved these issues and allowed the transformers to function correctly within cross-validation and pipeline cloning procedures.

Given an additional week, the next area of investigation would be regularised polynomial regression on the California Housing dataset. Specifically, a comparison between Ridge regression with polynomial features and the existing unregularised degree-2 OLS model would provide insight into the trade-off between increased model flexibility and overfitting control. Such an analysis could determine whether regularisation improves predictive performance while retaining the benefits of nonlinear feature interactions.

The weakest component of the current codebase is the `test_preprocessing.py` file, which remains effectively empty. As a result, `pytest` currently has no preprocessing tests to execute, despite testing being an explicit requirement of the assignment. Expanding this test suite to verify the correctness and robustness of the preprocessing components would improve software quality and strengthen the overall project submission.

A. Appendix – Additional figures or tables (if any)

[Optional. Place any figures or tables that didn't fit in the main parts here, with clear labels and a sentence of context for each.]

End of Homework 3 report.